INSTITUTO SUPERIOR TÉCNICO

Parallel and Distributed Computing
Parsim MPI Report

João Gouveia - 102611, Rodrigo Arede - 102606, Gonçalo Rua - 102604
April 4, 2025

1 Introduction

For this stage of the project we explored paralellizing the particle simulator using MPI and OpenMP. We developed several solutions, and empirically compared them using RNL's cluster.

2 Approach

2.1 Lines

Our first approach was dividing the grid in lines, and assigning each line to a process. This is by far the easiest approach, it is also, at least theoretically, the worst.

2.2 Blocks

To minimize communication between nodes, we experimented with splitting the grid into blocks. Each task first computes the centers of mass for the cells that need to be communicated and sends them to the appropriate rank. While waiting to receive the centers of mass it requires, it computes the centers of mass for the cells that do not need to be communicated. The process for calculating particle positions is analogous.

Block dimensions are computed dynamically. Let $ncside$ denote the size of the grid's sides, $nprocs$ the number of tasks available to us, and $npart$ the total number of particles. Let $size = \max(ncside/2, 5)$ denote the height of our blocks. Each cell has a weight corresponding to the number of particles assigned to it, and each rank has a weight equal to the sum of the weights of the cells assigned to it. While the rank weight is less than a threshold value, defined as $\max(npart/nprocs, 10)$, we continue assigning vertical lines of $size$ cells to the current rank. Once this threshold is exceeded, we advance to the next rank in a round-robin fashion.

2.3 Blocks And OpenMP

After fine-tuning the parameters of our block strategy, we used OpenMP to parallelize the loop that calculates the new particle positions. This further reduced the running time of our program.

3 Synchronization Concerns

During the main loop, there is only one point where synchronization is required: after calculating the centers of mass of all the particles, each rank must wait to receive the centers of mass of the cells surrounding its block. Besides this, we use OpenMP's `critical` directive to synchronize access during the parallelized loop.

4 Load Balancing

Our block construction inherently balances the load by dynamically adjusting the size of the blocks to achieve a more homogeneous distribution of particles across ranks. Additionally, we use OpenMP's dynamic scheduling in our parallelized loop.

5 Performance Results

5.1 Methodology

We tested our solution multiple times, using different numbers of tasks to refine our strategies for block calculation and load balancing. The following tests were used:

Test Number	Input
1	12672 0.05 3 10 10
2	5893 0.05 3 10 10
3	8555 0.05 3 10 10
4	12 100 5 10000 10000
5	-11 3500 20 500000 10
6	1 5000 100 1000000 4
7	1 5000 100 1000000 100
8	1 5000 20 1000000 10
9	1 1000 3 10000 10000
10	3 5000 50 1000000 300
11	3 5000 50 1000000 500
12	-1 1000 30 100000 1000

5.2 Results

The table below shows the execution times (in seconds) for the serial and OpenMP versions of the program:

| Test Number | 1 thread (serial) | 2 threads | 4 threads | 8 threads | |————-|————————-|————|————|————| | 1 | 0.0s | 0.0s | 0.0s | 0.0s | | 2 | 0.0s | 0.0s | 0.0s | 0.0s | | 3 | 0.0s | 0.0s | 0.0s | 0.0s | | 4 | 53.9s | 31.4s | 21.2s | 16.7s | | 5 | 49.7s | 25.5s | 13.6s | 7.5s | | 6 | 1.6s | 1.0s | 0.8s | 0.8s | | 7 | 34.9s | 23.0s | 18.1s | 15.7s | | 8 | 57.1s | 29.7s | 15.6s | 8.7s | | 9 | 245.9s | 141.4s | 89.8s | 68.0s | | 10 | 289.4s | 160.5s | 100.1s | 74.1s | | 11 | 481.9s | 263.0s | 171.8s | 128.5s | | 12 | 81.8s | 45.7s | 30.4s | 24.0s |

6 Conclusion

The project successfully implemented a parallel particles simulation, demonstrating significant speedups using OpenMP. The next step is to use MPI to parallelize the workload across different machines, and exploring an hybrid MPI+OpenMP approach for even better performance and scalability.